COMPLETE BUILD GUIDE

2024 Edition

FROM ZERO TO LIVE

How to Build a 3D Interactive Website

A clear, detailed guide that covers everything you need — the right tools, how 3D scenes work, adding real characters with animations, making the world feel alive, and deploying for free. No prior 3D experience needed.

Three.js & WebGL

3D Characters & Animation

Living Scene Design

Performance Tuning

Free Deployment

Mobile Friendly

Complete Field Guide

Architecture · Tools · Best Practices · Code Concepts

6	30+	$0
CHAPTERS	CONCEPTS	COST TO START

FOUNDATION

1 Understanding 3D on the Web

How Does 3D Work Inside a Browser?

Every modern browser has a built-in 3D graphics system called **WebGL**. WebGL speaks directly to your device's graphics card (GPU) and can render millions of 3D triangles every second — all inside a web page, with no plugins or installs.

However, writing raw WebGL code is extremely difficult — like writing in machine language. So developers use a JavaScript library called **Three.js** which sits on top of WebGL and makes building 3D scenes practical and readable.

What is WebGL?

- Built into Chrome, Firefox, Safari, Edge — all browsers
- Talks directly to the GPU — fast, hardware-accelerated
- Renders at up to 60 frames per second
- No installation — runs as pure JavaScript
- Supported on 97%+ of all devices worldwide

What is Three.js?

- The most popular 3D library for the web (100K+ GitHub stars)
- Wraps WebGL into clean, simple JavaScript code
- Geometry, materials, lights, cameras, animations — all built in
- Used by Google, NASA, Apple, game studios worldwide
- Free, open source, actively maintained for 12+ years

💡 **The Key Point**

You do not need Unity, Unreal Engine, or any paid software. A complete, professional 3D website runs entirely in JavaScript inside a browser. The same language used to build any website.

The Six Core Concepts — How Every 3D Scene Works

Before writing any code, understand these six ideas. Every 3D project — simple or complex — uses all of them.

1

Scene

The "world container." Every object, light, and character lives inside the Scene. Think of it as the stage in a theatre.

2

Camera

The viewer's eye. The camera decides what you see, from which angle, and with what zoom. Moving the camera is how you navigate.

3

Renderer

Takes the Scene + Camera and draws it as pixels on screen. Runs 60 times per second. This is where the GPU does its work.

4

Mesh

Any 3D object = Geometry (its shape) + Material (its surface — color, roughness, transparency). Combined, that is a Mesh.

5

Light

Without light, everything is black. Add a directional light (sun), hemisphere light (sky), and fill light to illuminate the scene.

6

Animation Loop

A function that runs every frame (~60fps). It updates positions, plays animations, then calls the renderer. This is what makes things move.

Regular Website vs 3D Website — The Real Differences

Aspect	Regular Website	3D Website
Rendering	Browser draws HTML and CSS elements	GPU renders a 3D scene onto a canvas element
Layout	Flat 2D — Flexbox, Grid, positions	3D space — X, Y, Z coordinates
Interaction	Click, hover, scroll	Drag to rotate, pinch to zoom, click 3D objects
Assets	Images, SVGs, fonts	3D models (.glb), textures, animation clips
Performance focus	DOM repaints, network speed	Frame rate, polygon count, GPU memory
Development time	Days to weeks	Weeks to months (but worth it)
Impression on visitors	Good — expected	Exceptional — remembered and shared

TOOLS & STACK

2 Choosing the Right Tools

Which 3D Library Should You Use?

Several options exist for building 3D websites. Here is an honest comparison.

Library	Best For	Difficulty	File Size	Community
Three.js	Custom 3D experiences, full creative control	Medium	~600 KB	Very large
Babylon.js	Games, physics, larger teams	Medium-High	~2 MB	Large
React Three Fiber	React apps that need 3D elements	Medium (requires React)	600 KB+	Large
A-Frame	VR / AR, quick HTML-based prototypes	Low	~900 KB	Medium
Spline	Simple embeds, no-code approach	Very Low	Large	Medium

✅ **Recommendation — Start with Three.js**

Three.js gives you the most control, the best documentation, the most tutorials online, and works with plain JavaScript. No framework required. It is the industry standard for custom 3D web experiences.

3D Modeling Software — Creating Your Assets

3D objects (buildings, furniture, characters) are designed in separate software and then exported as files. You do not build complex shapes in code — you create them visually, then load them into your website.

🟠 Blender — Recommended (Free)

- **Cost:** 100% free, open source
- **Use for:** Buildings, furniture, props, environments, vehicles
- **Export:** .glb format — Three.js reads this natively
- **Time to learn:** 2–4 weeks of basics from YouTube tutorials
- **Why:** Industry-grade quality, massive tutorial community

🟣 Mixamo — Free Characters

- **Cost:** Free (needs an Adobe account)
- **Use for:** Ready-made human characters with animations
- **Animations included:** 2,000+ clips — walk, run, sit, wave, talk
- **Export:** FBX → convert to .glb using Blender
- **Why:** No rigging skills needed — characters ready in minutes

For Textures (Surfaces)

- **polyhaven.com** — free PBR textures
- **ambientcg.com** — free seamless materials
- **Substance Painter** — professional tool
- Always compress before use on the web

For Free 3D Models

- **Sketchfab** — huge library, filter by license
- **Quaternius on itch.io** — free low-poly packs
- **KenneyNL** — free game-ready assets
- Always check license before using

Local Dev Server

- Never open index.html directly — JS modules require a server
- **Node.js:** `npx serve .` or write a small server
- **Python:** `python3 -m http.server`
- **VS Code:** Live Server extension

File Formats — What Goes Where

Format	What It Contains	When to Use
`.glb`	3D model + textures + animations, binary (compact)	✅ Your primary format for Three.js — always prefer this
`.gltf`	Same as glb but stored as readable JSON + separate files	When you need to inspect or manually edit the raw data
`.fbx`	Autodesk format, good for Mixamo animation downloads	Source file only — convert to .glb before using on the web
`.png / .jpg / .webp`	Texture images — color, roughness, normal maps	Surface detail painted on materials

> ⚠️ **Always Optimize Assets Before Publishing**

A raw Blender file can be 50 MB. Web visitors cannot wait for that. Use **gltf-transform** (a free command-line tool) to compress your models. Target a total scene size under 10 MB. Compress textures to WebP format. This single step often cuts load time by 70%.

ARCHITECTURE

3 Designing and Building Your 3D Scene

How to Organize Your Project Files

A clean file structure is not optional — it is essential. The larger your scene gets, the more you will rely on good organization to stay sane.

RECOMMENDED FOLDER STRUCTURE

```
my-3d-website/
├── index.html           ←  Minimal HTML shell — just a <canvas> and script tags
├── main.js              ←  Entry point — sets up scene, camera, renderer, starts the loop
├── config.js            ←  All user settings — colors, text, names — kept separate and
clean
│
├── scene/
│   ├── environment.js   ←  Sky, lighting, fog, atmosphere
│   ├── architecture.js  ←  Buildings, floors, walls — the structural shell
│   ├── furniture.js     ←  Chairs, desks, props, decorations
│   └── neighbourhood.js ←  Outdoor world — roads, buildings, nature
│
├── characters/
│   ├── loader.js        ←  Loads .glb character files and their animations
│   ├── controller.js    ←  Handles movement, states (idle/walk/sit)
│   └── npc.js           ←  Non-player characters and their routines
│
├── systems/
│   ├── camera.js        ←  Camera controls, animated transitions between views
│   ├── routes.js        ←  Waypoints and pathfinding for characters
│   ├── traffic.js       ←  Vehicle routing and junction logic
│   └── performance.js   ←  Quality profiles, device capability detection
│
├── assets/
│   ├── models/          ←  Compressed .glb files
│   └── textures/        ←  .webp image files
│
└── vendor/
    └── three.module.js  ←  Three.js stored locally — no CDN needed
```

📌 **Split by Responsibility, Not by Type**

Do not put all 3D code in one file. Split by what each piece does — environment, characters, UI, systems. A real 3D project grows to thousands of lines quickly. Good structure prevents it from

becoming unmanageable.

The Core Rendering Loop — The Heartbeat of Every 3D App

This is the most important pattern to understand. It runs roughly 60 times per second and is the foundation of your entire application.

```
// Step 1 — SETUP (runs only once when the page loads)
const scene    = new THREE.Scene();          // Create the world
const camera   = new THREE.PerspectiveCamera(60, width / height, 0.1, 1000);
const renderer = new THREE.WebGLRenderer({ antialias: true });
renderer.setSize(width, height);
document.body.appendChild(renderer.domElement); // Add canvas to the page

// Step 2 — ANIMATION LOOP (runs ~60 times per second)
renderer.setAnimationLoop(() => {
  controls.update();            // Update camera from mouse or touch input
  updateCharacters(delta);      // Move all characters forward by delta time
  updateVehicles(delta);        // Move all vehicles forward
  renderer.render(scene, camera); // Draw the current frame
});
```

Three Ways to Build 3D Objects

A **Procedural Geometry — Build Shapes Directly in Code**

Create boxes, cylinders, and spheres directly using Three.js geometry classes. Very fast to iterate, tiny file size, no external files. Perfect for architecture — walls, floors, desks, and partitions built from simple shapes.

Maximum control | Tiny file size

B **Loaded Models — Import from Blender**

Design in Blender, export as .glb, load with Three.js `GLTFLoader`. Best for complex organic shapes — trees, cars, detailed furniture, characters.

Most realistic results | Larger file sizes

C **Hybrid — The Best Approach for Real Projects**

Use procedural code for the architectural shell (walls, floors, structural elements) and load .glb files only for complex details (characters, trees, vehicles). This gives the smallest total size with the best visual quality.

✅ Recommended

Lighting — The Difference Between Flat and Beautiful

Poor lighting makes a great scene look amateur. Good lighting makes a simple scene look stunning. Use this three-light setup as your starting point:

☀️ Directional Light (Sun)

- Simulates sunlight — parallel rays from one direction
- Enable shadows on this light only
- Position high and at an angle (like a real sun)
- Use a warm color — #ffefd7 looks natural
- Keep intensity around 1.8 to 2.5

🌤️ Hemisphere Light (Sky)

- Sky color from above, ground color from below
- No shadows — purely ambient fill
- Prevents pitch-black shadow areas
- Always add this — single biggest quality jump
- Sky: warm white, Ground: muted green/grey

💡 Fill Light (Secondary)

- A second directional light from the opposite side
- Low intensity — around 0.4 to 0.6
- Cool or bluish color to contrast the warm sun
- Adds dimensionality — objects don't look flat
- No shadows needed on this light

LIFE & MOVEMENT

4 Characters, Animation & Living Scenes

Adding 3D Characters — The Mixamo Workflow

Building a human character from scratch in 3D takes months of specialized work. The practical solution is **Mixamo** — Adobe's free library of ready-made rigged characters with thousands of motion capture animations.

1. **Pick a Character** — mixamo.com — choose any human model from the library
2. **Download Animations** — Walk, Idle, Sit, Wave — download each as FBX separately
3. **Combine in Blender** — Import all FBX files, merge animation clips into one .glb
4. **Load in Three.js** — GLTFLoader + AnimationMixer — play and blend clips
5. **Write a State Machine** — Control which animation plays based on what the character is doing

```
// Loading a character and playing animations in Three.js
const loader = new GLTFLoader();
loader.load('./assets/character.glb', (gltf) ⇒ {
  scene.add(gltf.scene);

  // AnimationMixer controls all animations for this character
  const mixer = new THREE.AnimationMixer(gltf.scene);
  const idleAction = mixer.clipAction(gltf.animations[0]);
  const walkAction = mixer.clipAction(gltf.animations[1]);

  idleAction.play(); // Start with idle animation

  // To switch to walking — smooth 0.3 second crossfade
  idleAction.crossFadeTo(walkAction, 0.3, true);
  walkAction.play();

  // Call this every frame inside your animation loop
  mixer.update(deltaTime); // deltaTime = seconds since last frame
});
```

State Machine — How Character Behavior is Controlled

A character's behavior is driven by a **state machine** — a list of possible states (Idle, Walking, Sitting, etc.) with defined rules for when to switch between them. This is how you make NPCs behave believably without writing complex AI.

Example States — Office Receptionist

- **IDLE** — Sitting at desk, subtle breathing animation
- **GREETING** — Looks at visitor, plays wave animation
- **TYPING** — Head down, keyboard animation active
- **STANDING** — Transition from seated to standing
- **ESCORTING** — Walking beside the visitor to a room
- **RETURNING** — Walking back to the desk after escort

Waypoint Routing — Simple & Effective

- Define waypoints as X, Z coordinates in 3D space
- A route is an ordered list of waypoints
- The character moves toward the current waypoint
- When it gets close enough → advance to the next one
- At the final waypoint → trigger the next state
- Simple, reliable — no pathfinding AI needed

Making the World Feel Alive — Living Scene Design

A scene with only static objects feels like a photograph. A *living scene* has continuous movement, rhythm, and activity that makes it feel like a real place. Here is what to add:

🚗 Vehicle Traffic

- Define lane paths as curves or arrays of points
- Cars follow the path at varied, randomized speeds
- Simple yielding logic at junctions
- Randomize car colors and types for variety
- Respawn vehicles off-screen so the road feels busy

🚶 Pedestrians

- Give each pedestrian a looping circuit to walk
- Randomize pause duration at "stop points"
- Vary walking speeds across characters
- Keep pedestrians off vehicle roads — use pavements
- Different clothing colors create visual variety

✈️ Atmosphere Events

- Airplane crossing overhead every 60–90 seconds
- Boats following a river path continuously
- Train running a loop between two stations
- Foliage swaying via a simple wind shader
- These tiny details make visitors say "wow"

🎯 **The Secret to a Scene That Feels Real**

Living scenes feel alive because of **rhythm and randomness working together**. The airplane arrives predictably every 70 seconds (rhythm) but boats move at slightly different speeds (randomness). Pedestrians follow fixed circuits (rhythm) but pause for random durations (randomness). This combination is exactly what the human brain finds satisfying and believable.

PERFORMANCE & UX

5 Performance, UI Design & User Experience

Why Performance is Not Optional in 3D

A slow 3D website does not just frustrate users — it physically heats up their device, drains battery, and can crash a browser tab on older phones. You must measure and control performance from the start, not as an afterthought.

Metric	Target	What Happens If Too High
Frame Rate	60 fps desktop, 30 fps mobile minimum	Scene stutters and feels broken
Draw Calls	Under 300 per frame	CPU becomes the bottleneck — GPU sits idle waiting
Triangle Count	Under 500,000 visible at once	GPU overloads — frame rate drops sharply
Texture Memory	Under 200 MB total	Runs out of GPU VRAM — crashes on mobile devices
Initial Load Time	Under 5 seconds on 4G	50% of visitors leave before the scene appears

Quality Profiles — One Experience, Every Device

A MacBook Pro and a budget Android phone cannot run the same scene settings. Build three quality profiles and detect the device automatically.

🟢 Lightweight

- Shadows: off
- Pixel ratio: locked to 1x
- Anti-aliasing: off
- Fewer characters in the scene
- Simpler materials, no reflections
- **For:** old phones, low RAM devices

🟡 Balanced

- Shadows: basic (1024px shadow map)
- Pixel ratio: capped at 2x
- Anti-aliasing: on
- Full character count
- Standard materials
- **For:** most users — this is your default

🔵 High

- Shadows: soft PCF (2048px shadow map)
- Pixel ratio: native device DPR
- Anti-aliasing: MSAA
- Full scene, reflections, effects
- Environment map for reflective surfaces
- **For:** gaming laptops, M-series Macs

```
// Detect device capability and choose the right quality profile automatically
function detectQuality() {
  const ram      = navigator.deviceMemory;   // RAM in GB — Chrome/Edge only
  const isMobile = /Mobi|Android/i.test(navigator.userAgent);
  const isLowRAM = ram && ram <= 4;

  if (isMobile && isLowRAM) return 'low';      // Weak phone
  if (isMobile)             return 'balanced'; // Average phone
  return 'high';                               // Desktop / laptop
}
```

Six Critical Performance Techniques

Instanced Mesh — Repeat Objects Efficiently

If you have 50 identical trees or 30 chairs, do not create 50 separate objects. Use `InstancedMesh` — it renders them all in a single draw call. Can give a 50x performance improvement for repeated objects.

Share Materials — Never Duplicate

Never create the same material twice. If 100 boxes all use the same green color, they should all reference **one** shared Material object. Use a simple Map as a material cache to enforce this.

Pause When Hidden — Save Battery

Stop the animation loop when the browser tab is not visible. Use the `document.visibilitychange` event. Resume when the user comes back. Simple to add — saves significant battery on mobile.

Frustum Culling — Skip What You Cannot See

Three.js automatically skips rendering objects outside the camera's view. Help it by keeping `frustumCulled = true` on all objects (the default). Group distant objects together so they can be culled as one unit.

LOD — Less Detail at Distance

Show a high-polygon model when close, a low-polygon version when far away. Three.js has a built-in `LOD` class for this. In a 3D city, distant buildings can be simple colored boxes while nearby ones are fully detailed.

Shadow Map Budget

Only enable shadows on your main directional light. Keep shadow map size at 1024×1024 — going to 2048+ quadruples the cost. Limit the shadow camera frustum to cover only the visible area, not the whole scene.

UI Design — The 2D Layer on Top of 3D

Your 3D canvas is the background. All menus, info panels, buttons, and labels are normal HTML and CSS, positioned on top using `position: absolute`. The two layers work independently — the 3D scene renders on a canvas, HTML UI floats above it.

HTML UI Layer (Overlaid on 3D)

- Room navigation — clickable button list
- Info panels — descriptions, questions, prompts
- Loading screen with progress bar
- Toolbar — zoom, quality toggle, view reset
- Character name labels positioned above heads
- Error messages and status indicators

Clicking 3D Objects — Raycasting

- On mouse click, cast a ray from the camera through the cursor position
- Three.js checks which 3D object the ray intersects first
- Use `raycaster.intersectObjects()` to get the result
- On hit → trigger an action (open a panel, move the camera)
- This is how all 3D "clickable" objects work on the web

🎨 **Design Tip — Glassmorphism Works Perfectly for 3D UI**

Use `backdrop-filter: blur(16px)` with a semi-transparent background color for all UI panels. This lets the 3D scene show through without competing with the text. It creates a sense of depth — as if the panel is floating inside the world. This is the standard look for modern 3D web interfaces.

SHIP IT

6 Deployment, Hosting & Going Live

The Big Advantage — It Is Just Static Files

Unlike a game that needs dedicated servers, a Three.js 3D website is just **static files** — HTML, JavaScript, CSS, and 3D assets. No database, no server-side code, no DevOps. You can host it anywhere for free, with zero server maintenance.

☁️ Cloudflare Pages

- Best performance — global CDN edge network
- Free tier: unlimited requests and bandwidth
- Deploy from GitHub in under 30 seconds
- Free custom domain connection + SSL
- Supports custom headers file for caching

🟢 Netlify

- Easiest to start — drag and drop your folder to deploy
- Free tier: 100 GB bandwidth per month
- Instant deploy from GitHub or GitLab
- Free custom domain + automatic SSL
- Great choice for beginners

🐙 GitHub Pages

- Free for public repositories — permanently
- Deploy directly from any GitHub repository
- Gets a username.github.io domain instantly
- Slightly slower CDN than Cloudflare or Netlify
- Best for personal portfolios and demos

HTTP Headers — Critical for 3D Asset Performance

3D models and textures are large files. With correct caching headers, a returning visitor loads your site in under 1 second because the assets are already stored in their browser. Add a `_headers` file to your project root (Cloudflare and Netlify read it automatically):

```
# _headers — place this file in your project root folder

/*
  X-Content-Type-Options: nosniff
  X-Frame-Options: SAMEORIGIN
  Referrer-Policy: strict-origin-when-cross-origin

# Cache 3D models and textures for 1 year — they do not change between visits
/assets/*
  Cache-Control: public, max-age=31536000, immutable

# Cache the Three.js vendor file — it never changes
```

```
/vendor/*
  Cache-Control: public, max-age=31536000, immutable

# Never cache HTML — so visitors always get your latest updates instantly
/*.html
  Cache-Control: no-cache, no-store, must-revalidate
```

Pre-Launch Checklist

Category	What to Check	How to Verify
Assets	All .glb files compressed with gltf-transform	Total asset folder under 10 MB
Performance	60 fps on a mid-range laptop	Chrome DevTools → Performance tab → record a session
Mobile	Works on iOS Safari and Android Chrome	Test on a real physical device, not just the DevTools emulator
Loading	Progress bar visible while assets load	Throttle to "Slow 3G" in DevTools → reload the page
Error handling	Graceful message if WebGL fails	Disable hardware acceleration in browser settings → reload
Accessibility	All UI buttons navigable by keyboard	Tab through every button — check for visible focus indicator
SEO	Title, description, and Open Graph tags set	View page source → look for meta tags in the <head>
HTTPS	Site loads over HTTPS only	Browser shows a padlock icon in the address bar

Your Learning Roadmap — From Beginner to Live

1. **Weeks 1–2** — Three.js basics — scene, camera, lights, simple shapes, animation loop
2. **Weeks 3–4** — Blender basics — model a simple room, export .glb, load in Three.js
3. **Weeks 5–6** — OrbitControls, GLTFLoader, HTML UI overlay, raycasting for clicks
4. **Weeks 7–8** — Mixamo characters, AnimationMixer, state machine for NPC behavior
5. **Weeks 9–10** — Living scene — vehicles, pedestrians, atmosphere events, routing
6. **Weeks 11–12** — Optimize assets, test on real devices, deploy to Cloudflare Pages

Best Free Resources to Learn

Three.js

- **threejs.org/docs** — official, always current
- **Bruno Simon — Three.js Journey** (paid but worth it)
- **discoverthreejs.com** — free, excellent beginner guide
- **YouTube: Fireship, WaelYasmina** — short tutorials

Blender

- **Blender Guru — Donut Tutorial** (YouTube, the classic start)
- **Grant Abbitt** — low-poly modeling tutorials
- **blender.org/support/tutorials** — official resources
- **CG Cookie** — structured courses

Free 3D Assets

- **polyhaven.com** — HDRIs, textures, models
- **sketchfab.com** — huge 3D model library
- **mixamo.com** — characters and animations
- **quaternius.com** — free low-poly packs

Tools

- **gltf-transform** — optimize and compress .glb files
- **gltf.report** — inspect your model in the browser
- **squoosh.app** — compress textures to WebP
- **VS Code + Live Server** — local dev setup

FINAL THOUGHT

A 3D website is not a trend. It is a competitive advantage.

In a world of identical landing pages and template-built portfolios, a living 3D experience is remembered. It communicates craftsmanship, technical depth, and genuine effort — before a single word is read. The tools are free. The knowledge is learnable. The differentiation is real and lasting.